

# Algorithm & Computational Intelligence Documentation

## Mathematical, Cryptographic, Spatial & Machine Learning Specifications

|                  |                                                  |                    |                                                                        |
|------------------|--------------------------------------------------|--------------------|------------------------------------------------------------------------|
| Document ID:     | DOC-ALG-2026-V1                                  | Aligned Standards: | ISO/IEC 27037:2012, NIST SP 800-86, FRE 902(14)                        |
| Creator & Owner: | <b>Timon Biswas</b><br>(timonbiswas33@gmail.com) | Classification:    | <b>Proprietary Architecture &amp; Inventions</b>                       |
| Audit Status:    | <b>11/11 Verified Implemented (100% Pass)</b>    | Anti-Cloning:      | Public inspection allowed; cross-language cloning strictly prohibited. |

### 1. Executive Overview & Anti-Cloning Invariant

FORENZA is a zero-trust forensic evidence custody and integrity verification platform designed by Timon Biswas. Under strict evidentiary requirements (Federal Rules of Evidence Rule 902(14) and ISO/IEC 27037:2012), digital records must be mathematically defensible. FORENZA replaces vulnerable administrative databases with deterministic cryptographic Merkle state DAGs and asymmetric hardware signatures, while utilizing multi-model AI strictly in an assistive capacity.

**Core Architectural Invariant:** Deterministic cryptography is the immutable root of trust. AI never computes or modifies evidence hashes, Ed25519 signatures, or state DAG nodes. The platform is source-available for public inspection and research verification; replicating or cloning this architecture in any programming language without written consent is strictly prohibited.

2. Actual Algorithm Inventory (11 Verified Implemented)

| Algorithm / Method      | Status      | Verified Source Location                     | Complexity & Purpose                                                     |
|-------------------------|-------------|----------------------------------------------|--------------------------------------------------------------------------|
| RFC 8785 JCS            | IMPLEMENTED | web/lib/crypto/canonical.ts                  | O(K log K) — Deterministic JSON key canonicalization                     |
| SHA-256                 | IMPLEMENTED | web/lib/crypto/evidence-hash.ts              | O(L) — 256-bit cryptographic digest & state pointers                     |
| Recursive State DAG     | IMPLEMENTED | web/lib/state/evidence-state-engine.ts       | O(N) — Merkle chronological state chain (E0 -> E1 ...)                   |
| Ed25519 (EdDSA)         | IMPLEMENTED | web/lib/crypto/signatures.ts                 | O(1) — Curve25519 hardware key signing & non-repudiation                 |
| AES-256-GCM             | IMPLEMENTED | web/lib/vault/offline-vault.ts               | O(L) — Authenticated encryption for offline vault media                  |
| Haversine Distance      | IMPLEMENTED | web/lib/geofence.ts                          | O(1) — Great-circle 500m crime scene geofence check                      |
| FZ-EPRA & FZ-DIV        | IMPLEMENTED | web/lib/reconciliation/epra.ts               | O(min(NA,NB)) — First divergence detection & dual-history reconciliation |
| Monotonic Sync Guard    | IMPLEMENTED | web/lib/sync/sync-engine.ts                  | O(1) — Rollback and replay attack prevention in sync                     |
| Cosine Similarity       | IMPLEMENTED | web/lib/ai/pipelines/case-search-pipeline.ts | O(K*d) — High-dimensional case-isolated vector RAG                       |
| Prompt Injection Filter | IMPLEMENTED | web/lib/ai/context-builder.ts                | O(M) — XML boundary delimiter isolation & token defense                  |
| HMAC-SHA256 Token       | IMPLEMENTED | web/lib/tokens/qr-token.ts                   | O(1) — Time-limited single-use QR custody transfer nonces                |

3. Detailed Algorithm Specifications & Proofs

|                                                                                                                                                                                                                                               |                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 3.1 RFC 8785 JSON Canonicalization Scheme (JCS)                                                                                                                                                                                               | VERIFIED IMPLEMENTED |
| Source Location: web/lib/crypto/canonical.ts -> canonicalizeJson(obj)                                                                                                                                                                         |                      |
| Mathematical Formula:                                                                                                                                                                                                                         |                      |
| $JCS(0) = \{ k_{(0)}: JCS(v_0), \dots, k_{(m-1)}: JCS(v_{m-1}) \} \text{ where } k_{(0)} < \dots < k_{(m-1)} \text{ in UTF-16 code units}$                                                                                                    |                      |
| <b>Operational Specification:</b> Standardized Unicode code-point key sorting ensuring that semantically identical JSON manifests generate identical byte sequences across different programming environments (TypeScript, Rust, Python, Go). |                      |
| <b>Complexity &amp; Role:</b> O(K log K + N) Time   O(N) Space — <i>Guarantees cross-platform evidence hash identity under ISO/IEC 27037.</i>                                                                                                 |                      |

|                                                                                                                                                                                                                |                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 3.2 Secure Hash Algorithm 256 (SHA-256)                                                                                                                                                                        | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/crypto/evidence-hash.ts -&gt; sha256(), sha256Bytes()</code>                                                                                                                    |                      |
| Mathematical Formula:                                                                                                                                                                                          |                      |
| $H_{master} = \text{SHA-256}(\text{JCS}(\text{Manifest})), \text{ where } \text{SHA-256}(M) = \text{Compress}(H_{i-1}, M_i)$                                                                                   |                      |
| Operational Specification: Cryptographic one-way 256-bit collision-resistant digest function. Any alteration of a single bit produces a completely different avalanche hash output.                            |                      |
| Complexity & Role: $O(L)$ Time linear in byte length   $O(1)$ Space — <i>Provides legal defensibility and self-authentication under FRE Rule 902(14).</i>                                                      |                      |
| 3.3 Recursive Merkle State DAG Hashing (FZ-TWIN)                                                                                                                                                               | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/state/evidence-state-engine.ts -&gt; computeStateHash(), verifyStateHistory()</code>                                                                                            |                      |
| Mathematical Formula:                                                                                                                                                                                          |                      |
| $H_n = \text{SHA-256}(\text{JCS}(\{ H_{n-1}, H_{\text{event}, n}, \text{Actor}_n, \text{Device}_n, T_n, L_n \}))$                                                                                              |                      |
| Operational Specification: Incorporates the immediate parent state hash into the current state manifest, creating an unbroken Merkle lineage chain. Retroactive deletion breaks all subsequent state pointers. |                      |
| Complexity & Role: $O(N)$ Time for N states   $O(1)$ Space — <i>Ensures strict chronological custody ordering and tamper detection.</i>                                                                        |                      |
| 3.4 Ed25519 Elliptic Curve Digital Signatures (EdDSA)                                                                                                                                                          | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/crypto/signatures.ts -&gt; Ed25519Signer.sign(), verify()</code>                                                                                                                |                      |
| Mathematical Formula:                                                                                                                                                                                          |                      |
| $R = rB, S = (r + \text{SHA-512}(R    A    M) * a) \bmod l$                                                                                                                                                    |                      |
| Operational Specification: Asymmetric public-key digital signature scheme over Twisted Edwards curve Curve25519. Provides high-speed hardware signing and mathematical non-repudiation.                        |                      |
| Complexity & Role: $O(1)$ Constant Time (~0.2ms)   $O(1)$ Space (64-byte signature) — <i>Binds state additions to hardware-attested officer keys for legal non-repudiation.</i>                                |                      |
| 3.5 Authenticated Encryption with Associated Data (AES-256-GCM)                                                                                                                                                | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/vault/offline-vault.ts -&gt; OfflineVaultEngine.encryptMedia(), decryptMedia()</code>                                                                                           |                      |
| Mathematical Formula:                                                                                                                                                                                          |                      |
| $C = P \text{ xor } \text{AES-CTR}_K(IV), T = \text{GHASH}_H(\text{AAD}    C    \text{len}(\text{AAD})    \text{len}(C)) \text{ xor } E_K(IV_0)$                                                               |                      |
| Operational Specification: Combines AES-256 counter mode encryption with Galois GMAC authentication tag verification using 96-bit random initialization vectors (IVs).                                         |                      |
| Complexity & Role: $O(L)$ Time linear in payload bytes   $O(L)$ Space — <i>Secures offline evidence on mobile field devices and detects ciphertext bit-flipping.</i>                                           |                      |

|                                                                                                                                                                         |                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>3.6 Haversine Great-Circle Geodesic Distance Algorithm</b>                                                                                                           | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/geofence.ts -&gt; haversineDistance(), verifyGeofence()</code>                                                                           |                      |
| Mathematical Formula:                                                                                                                                                   |                      |
| $d = 2R * \arcsin(\sqrt{\sin^2(d\Phi i/2) + \cos(\phi i1)*\cos(\phi i2)*\sin^2(d\Lambda bda/2)})$                                                                       |                      |
| Operational Specification: Calculates great-circle distance between officer capture coordinates and crime scene perimeter origin on a spherical Earth (R = 6,371,000m). |                      |
| Complexity & Role: O(1) Constant Time   O(1) Space — <i>Mathematically validates officer physical presence within crime scene radius (500m default).</i>                |                      |

|                                                                                                                                                                                     |                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>3.7 Evidence Provenance Reconciliation Algorithm (FZ-EPRA / FZ-DIV)</b>                                                                                                          | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/reconciliation/epra.ts -&gt; ReconciliationEngine.findFirstDivergence(), reconcile()</code>                                                          |                      |
| Mathematical Formula:                                                                                                                                                               |                      |
| $i\_div = \min \{ i \text{ in } [0, \min( HA ,  HB )-1] \mid HA[i].state\_hash \neq HB[i].state\_hash \}$                                                                           |                      |
| Operational Specification: Performs linear dual-history traversal between offline mobile device states and server state trees to pinpoint first divergence nodes non-destructively. |                      |
| Complexity & Role: O(min(NA, NB)) Time   O(1) Space — <i>Resolves offline capture conflicts non-destructively without overwriting history.</i>                                      |                      |

|                                                                                                                                                                          |                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>3.8 Monotonic Sequence Rollback &amp; Replay Guard</b>                                                                                                                | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/sync/sync-engine.ts -&gt; OfflineSyncEngine.processSyncEvent()</code>                                                                     |                      |
| Mathematical Formula:                                                                                                                                                    |                      |
| $Valid(E) \Leftrightarrow (Seq\_payload > Seq\_known) \text{ and } (EventID \text{ not in Processed}) \text{ and } Ed25519Verify()$                                      |                      |
| Operational Specification: Monotonic state verification rejecting out-of-order, duplicate, or replayed synchronization payloads from compromised or out-of-sync devices. |                      |
| Complexity & Role: O(1) Time   O(M) Space for processed event ID cache — <i>Prevents replay attacks and state rollbacks during asynchronous synchronization.</i>         |                      |

|                                                                                                                                                                         |                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <b>3.9 Cosine Vector Similarity &amp; Semantic Retrieval</b>                                                                                                            | VERIFIED IMPLEMENTED |
| Source Location: <code>web/lib/ai/pipelines/case-search-pipeline.ts -&gt; CaseSearchPipeline.execute()</code>                                                           |                      |
| Mathematical Formula:                                                                                                                                                   |                      |
| $\cos(\theta) = (u \cdot v) / (  u  _2 *   v  _2) = \text{sum}(u\_i * v\_i) / (\sqrt{\text{sum } u\_i^2} * \sqrt{\text{sum } v\_i^2})$                                  |                      |
| Operational Specification: Computes normalized dot-product across high-dimensional vector embeddings strictly pre-filtered by case authorization (WHERE case_id = \$1). |                      |
| Complexity & Role: O(K * d) Time for K pre-filtered items   O(K) Space — <i>Enables secure semantic evidence search without cross-case data leakage.</i>                |                      |

|                                                                                                                                                                     |                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 3.10 Context Minimization & Prompt Injection Sanitization                                                                                                           | VERIFIED IMPLEMENTED |
| Source Location: web/lib/ai/context-builder.ts -> AIContextBuilder.sanitizeUntrustedText()                                                                          |                      |
| Mathematical Formula:                                                                                                                                               |                      |
| <code>Text' = Enclose(RegexSuppress(Text), '')</code>                                                                                                               |                      |
| Operational Specification: Strips sensitive operational secrets and isolates untrusted OCR/evidence text within unambiguous XML boundary tags before LLM injection. |                      |
| Complexity & Role: O(M) Time linear with text length   O(M) Space — Prevents hostile evidence documents from hijacking AI reasoning workflows.                      |                      |

|                                                                                                                                                                                 |                      |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 3.11 HMAC-SHA256 Token Signing & Handover Verification                                                                                                                          | VERIFIED IMPLEMENTED |
| Source Location: web/lib/tokens/qr-token.ts -> generateEvidenceQRToken(), verifyEvidenceQRToken()                                                                               |                      |
| Mathematical Formula:                                                                                                                                                           |                      |
| <code>HMAC(K, M) = SHA-256((K xor opad)    SHA-256((K xor ipad)    M))</code>                                                                                                   |                      |
| Operational Specification: Generates time-limited, single-use custody transfer tokens with cryptographic nonces for secure QR code custody handovers between physical officers. |                      |
| Complexity & Role: O(1) Constant Time   O(1) Space — Ensures physical chain-of-custody transfers cannot be replayed or forged.                                                  |                      |

4. Automated Unit & Integration Test Verification (112/112 Passed)

Every single algorithm documented in this specification has been audited and verified via automated test suites. The automated Vitest runner executed 19 test files covering 112 assertions with a 100% pass rate.

| Algorithm / Method              | Target Test File                                  | Assertion Result      |
|---------------------------------|---------------------------------------------------|-----------------------|
| RFC 8785 JCS & SHA-256          | __tests__/evidence-hash.test.ts                   | 13 / 13 PASSED (100%) |
| Ed25519 Curve25519 Signatures   | __tests__/phase1-crypto-device.test.ts            | 6 / 6 PASSED (100%)   |
| Recursive Merkle DAG State      | __tests__/phase2-state-custody.test.ts            | 2 / 2 PASSED (100%)   |
| AES-256-GCM Vault & Sync        | __tests__/phase5-offline-vault-sync.test.ts       | 2 / 2 PASSED (100%)   |
| Haversine Distance & Geofence   | __tests__/geofence.test.ts                        | 8 / 8 PASSED (100%)   |
| FZ-EPRA & FZ-DIV Reconciliation | __tests__/phase3-reconciliation-branching.test.ts | 3 / 3 PASSED (100%)   |
| Monotonic Sync & Replay Guard   | __tests__/offline-sync.test.ts                    | 3 / 3 PASSED (100%)   |
| Cosine Vector RAG Search        | __tests__/phase-ai-orchestrator.test.ts           | 16 / 16 PASSED (100%) |
| Prompt Injection Defense        | __tests__/phase-ai-orchestrator.test.ts           | 16 / 16 PASSED (100%) |
| HMAC-SHA256 Transfer Nonces     | __tests__/custody-chain.test.ts                   | 13 / 13 PASSED (100%) |